



AI Remote

Your AI coding sessions, in your pocket.

Browse and remote-control your Claude Code and Cursor chats from your phone — self-hosted, single-user, open source.

MIT License

Self-hosted

Python · FastAPI · SQLite

Installable PWA

What's inside

01	At a glance	What it is and who it is for
02	How it works	Architecture, sync cycle, job flow
03	Features	Everything it can do
04	Security	Design, controls, and honest limits
05	Installation	Quickstart, agent service, production
06	Enabling remote commands	Allow-list and permission profiles
07	Configuration reference	Environment variables
08	Under the hood	Stack, structure, testing, contributing

Read this first

AI Remote exists to run AI-agent commands on your own machine on your behalf. That is the feature — and the reason section 04 is the most important one in this document.

What is AI Remote?

AI Remote is a small self-hosted web app that mirrors the chat history of Claude Code and Cursor from your Mac to your phone — and lets you send follow-up prompts back.

AI coding agents do long-running work: refactors, test runs, investigations. You start them at your desk, then walk away. AI Remote lets you check what the agent did, read the full conversation, and nudge it forward with a new prompt — from anywhere, in a mobile browser.

2

tools supported: Claude
Code & Cursor

0

inbound ports opened
on your Mac

1

command to start:
`./run.sh`

MIT

licensed, no accounts,
no cloud

The three parts



Local agent

A small Python process on your Mac (a `launchd` service). Reads your local chat files, uploads changes, and executes jobs you queue from the phone.



Backend

A FastAPI service in Docker with a SQLite + FTS5 database. Stores the chat index, message cache, and job queue. Serves the web UI.



Phone UI

A server-rendered web app, installable as a PWA. Search, filter, read, copy, and send prompts to an allow-listed project.

Who it is for

- Developers who run Claude Code or Cursor on a Mac and want to check in on sessions while away from the keyboard.
- People comfortable running a small Docker service and reading a security model before exposing it.

Who it is not for

- **Teams.** There is one user and one shared API key — no accounts, roles, or tenant isolation.
- **Anyone who wants a hardened, zero-trust remote-execution service.** See section 04 for exactly what is and is not defended.

Architecture

The Mac never accepts incoming connections. The agent reaches out to the backend; the phone talks only to the backend.

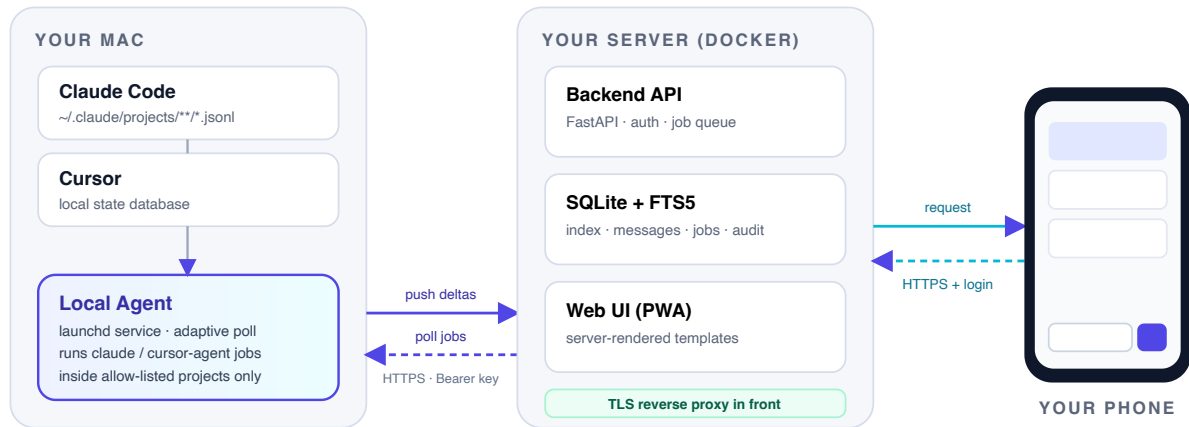


Figure 1 — Data flow. Both the agent and the phone connect to the backend; nothing connects to the Mac.

The sync cycle

Everything runs on a poll loop on the Mac. Each cycle the agent:

1. **Scans** Claude Code and Cursor session files and computes which sessions changed since the last sync.
2. **Pushes** only the deltas (session metadata and messages) to the backend.
3. **Polls** the backend for pending jobs.
4. **Executes** each job and **reports** the result; the next cycle then picks up the new content.

Adaptive polling: the backend tells the agent how often to poll — every **60 s** when idle, every **10 s** for 5 minutes after any authenticated request from the phone (sending a command, loading history, navigating, filtering). A header indicator shows the current mode.

Life of a remote command

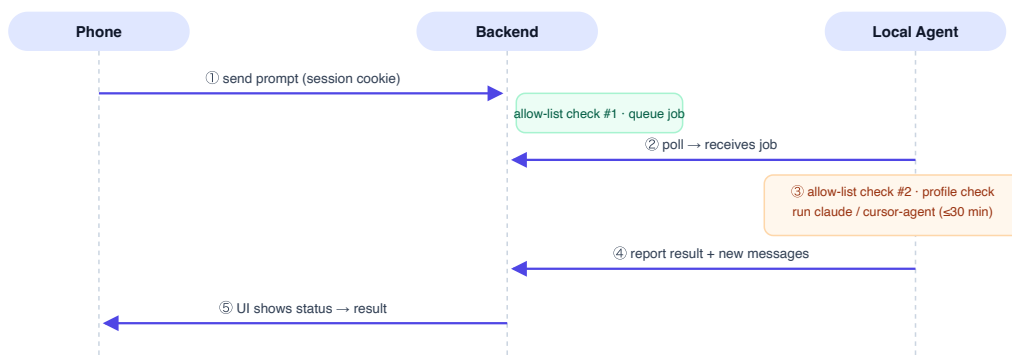


Figure 2 — A prompt is validated twice (backend and agent) before any CLI is invoked, and every job is recorded.

Resumed sessions run `claude -p --resume <id>` or `cursor-agent --resume <id> -p`; new sessions start a fresh headless run in the chosen project. Each job has a 30-minute subprocess timeout; if exceeded, it is killed and reported as `failed`.

What it can do

A focused feature set: read everything, send commands only where you explicitly allow it.

Read & search

- Browse **all** local Claude Code and Cursor sessions in one list.
- Filter by project path (with autocomplete), tool, and date group (today, yesterday, this week, older); sort by date, title, or path.
- **Full-text search** across sessions (SQLite FTS5).
- Formatted detail view: Markdown, tables, syntax-highlighted code blocks, clickable paths.
- **Load more** / **load full history** on demand, with an ETA countdown.
- Per-message **copy button** with rich-text clipboard support.
- English and German UI, light and dark theme, and a Settings page (language, theme, sync intervals, kill switch, logout).
- Mobile-first layout with a bottom tab bar; installable as a PWA with its own icon.

Control

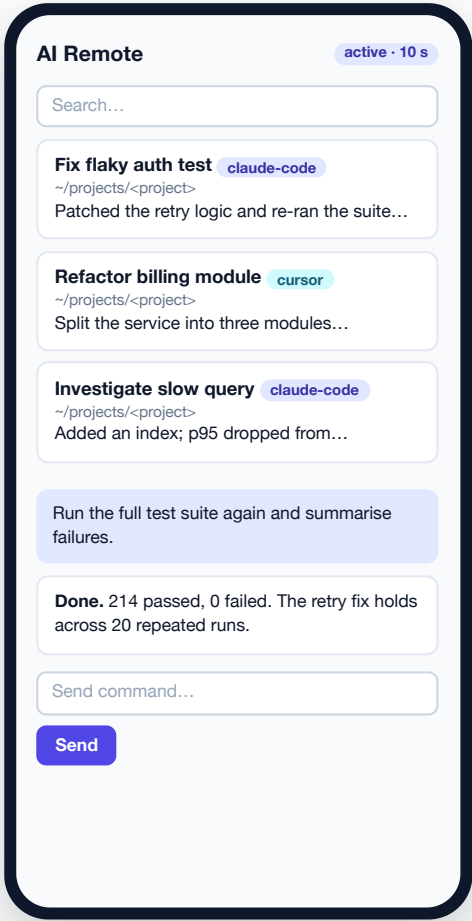
- **Continue** an existing session by sending a follow-up prompt.
- **Start a new session** in an allow-listed project.
- Optional voice input button for dictating prompts.
- Job status tracking from queued to running to done or failed.

Operate safely

- **Pause remote commands** kill switch — stops new jobs without affecting read-only sync.
- **Audit log** at `/jobs`: every job ever created, with type, target, prompt, status, and result.
- "Last reachable" indicator shows when the Mac last contacted the backend.

Feature at a glance

CAPABILITY	CLAUDE CODE	CURSOR	NOTES
Read & search history	Yes	Yes	Read-only; no changes to your local files
Continue a session remotely	Yes	Yes	Allow-listed projects with a valid permission profile only
Start a new session remotely	Yes	Yes	Same restrictions



Illustrative mock-up with sample data.

Security: by design, and honestly

AI Remote is secure *for what it is*: a personal tool with a deliberately small threat model. It layers several independent controls — and tells you plainly where the remaining sharp edges are.

The sharp edge, stated plainly

Anyone who holds the `API_KEY` can run AI-agent commands on your Mac — as your OS user, in auto-approving mode, inside your allow-listed projects. A leaked key therefore exposes everything that user can reach. Treat `API_KEY` like an SSH private key.

Defence in depth

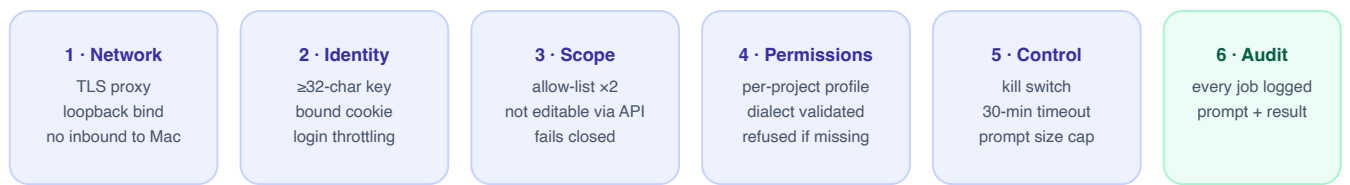


Figure 3 — A remote command must pass every layer before a CLI is ever invoked.

CONTROL	WHAT IT DOES
Outbound-only agent	The Mac opens no listening port. The agent only makes HTTPS requests to your backend.
Strong secrets, enforced	<code>API_KEY</code> and <code>SECRET_KEY</code> must be ≥32 characters; the backend refuses to start otherwise. Key comparison is constant-time.
Loopback by default	<code>docker-compose.yml</code> binds <code>127.0.0.1</code> . Exposure beyond localhost requires a TLS-terminating reverse proxy and <code>SESSION_COOKIE_HTTPS_ONLY=true</code> .
Login rate-limiting	Failed logins are throttled per client bucket (bounded memory, least-recently-active eviction). Deliberately no global lockout, so an attacker cannot lock <i>you</i> out from elsewhere.
Dual allow-list	<code>AI_REMOTE_ALLOWED_PROJECTS</code> is checked independently by backend and agent, set in two places, and is never editable through any API route. Empty means remote execution is off.
Enforced permission profiles	Each project needs a <code>.claude/settings.json</code> or <code>.cursor/cli.json</code> in the right tool dialect. Missing, empty, unparseable, or wrong-dialect profiles cause the job to fail <i>before</i> the CLI runs.
Sanitised rendering	Chat content is rendered from Markdown and passed through an HTML allow-list sanitiser (<code>bleach</code>).
Kill switch & audit log	Pause all remote execution with one tap; review every job ever created at <code>/jobs</code> .
Quiet by default	API docs (<code>/docs</code> , <code>/openapi.json</code>) are disabled unless <code>ENABLE_API_DOCS=true</code> .

Audited pre-release

A pre-release security audit found 0 Critical and 3 High issues, all in the remote-execution chain. All three are fixed, along with several Medium and Low findings.

Known limitations — read before you expose it

These are documented in `SECURITY.md`, each with a mitigation. They were left unfixed because fixing them would mean a design change out of proportion to a single-user tool.

LIMITATION	WHAT TO DO
One secret, two roles; sessions are stateless (14-day cookie, no per-session server-side revocation)	<code>/logout</code> ends your session; rotating <code>API_KEY</code> or <code>SECRET_KEY</code> invalidates all sessions. Don't let a browser remember the key.
No CSRF tokens (relies on <code>SameSite=Lax</code>)	Don't add a CORS policy or host untrusted content on sibling subdomains.
Permission profiles are deny-lists and templates are not exhaustive	Replace <code>allow</code> with a narrow explicit set per project.
Job history grows without bound	Back up and prune <code>data/app.db</code> yourself.
Login lock-out possible behind a proxy until configured	Set <code>TRUSTED_PROXY_HOPS</code> and <code>TRUSTED_PROXIES</code> together; keep the container reachable only via the proxy.
Dependencies are hash-pinned but not auto-audited	7-day release cooldown in <code>relock.sh</code> and Dependabot; run <code>scripts/audit-deps.py</code> before releases and review Dependabot PRs.
Container runs as root	Add a non-root <code>USER</code> and re-own the volume.
Reference deploy script SSHes as <code>root</code>	Deploy with a dedicated, minimally-privileged user.

Hardening checklist

- ① Generate both secrets with `openssl rand -hex 32`
- ② Keep the backend on loopback behind a TLS reverse proxy
- ③ Allow-list the smallest set of projects and read every permission profile line by line
- ④ Rotate both secrets if either was ever printed, pasted into a chat, or captured in a transcript.

Reporting a vulnerability: open an issue on the project repository, or contact the maintainer directly for anything sensitive. Please do not publish exploit details before a fix exists.

Get it running

Local in about two minutes; always-on in about ten.

Prerequisites

On your Mac

macOS, Python 3, `openssl`, and Claude Code and/or Cursor with some chat history.

For the backend

Docker with Compose. Locally that is your Mac; for production, a small server.

For internet access

A domain and a TLS-terminating reverse proxy (e.g. Nginx, Caddy, Plesk).

A · Quickstart (local)

1

Clone the repository

```
git clone https://github.com/<your-org>/ai-remote-management.git
cd ai-remote-management
```

2

Run the bootstrap script

```
./run.sh
```

On first run it creates `.env` from `example.env` with freshly generated secrets, starts the backend with Docker Compose, and runs one agent sync cycle so your real sessions appear straight away. It prints the login URL and the first characters of your API key; the full value is in `.env`. Re-running is safe and re-syncs.

3

Open the UI and log in

Visit the printed URL (by default `http://127.0.0.1:8000`) and log in with the `API_KEY` from `.env`.

4

Keep it syncing in the background

```
./setup-agent.sh
```

Installs the agent as a `launchd` service so syncing continues without a terminal. Re-runnable; remove with `./setup-agent.sh --uninstall`. Logs: `~/Library/Logs/ai-remote-agent.log`.

B · Deploy for phone access

To reach it from your phone, host the backend on a server behind HTTPS, then point the agent at it.

1

Configure `.env`

```
AI_REMOTE_BACKEND_URL=https://your-domain.example.com
SESSION_COOKIE_HTTPS_ONLY=true
SSH_HOST=user@your-server.example.com # for the SCP deploy path
```

2

Pick a deploy path

SCRIPT	HOW IT WORKS
<code>deploy-production-scp.sh</code>	Builds the image, ships it over SCP (no registry), restarts via SSH. Needs <code>SSH_HOST</code> and passwordless SSH.
<code>build-and-push.sh</code>	Builds and pushes to <code>REGISTRY</code> (e.g. <code>ghcr.io/<your-org></code>); deploy however your platform expects.
<code>deploy-to-plesk.sh</code>	The author's own Plesk Docker flow (SSH + <code>docker load</code>). Needs <code>PLESK_HOST</code> .

3

Terminate TLS in front of the container

Put a reverse proxy in front of port 8000. If you run behind one, set `TRUSTED_PROXY_HOPS` and `TRUSTED_PROXIES` *together* so login throttling sees real client addresses.

4

Re-run the agent setup against the public URL

```
./setup-agent.sh --backend-url=https://your-domain.example.com
```

5

On your phone

Open `https://your-domain.example.com`, log in, then use Safari's **Share** → **Add to Home Screen** to install the PWA.

Opt in, project by project

Reading history works out of the box. Sending prompts is **off** until you explicitly configure two things per project.

1 Allow-list the project path — on both sides

Set a comma-separated list of absolute, symlink-resolved paths in the repo-root `.env`, then re-run `./setup-agent.sh` so the value is baked into the agent's launchd service. The backend reads the same variable from its own `.env` on the server. The two copies are checked independently and never synced.

```
AI_REMOTE_ALLOWED_PROJECTS=/Users/you/projects/my-app,/Users/you/projects/other-app
# get the canonical form with: cd <dir> && pwd -P
```

2 Give each project a permission profile in the right dialect

TOOL	FILE	RULE TYPES
Claude Code	<code><project>/claude/settings.json</code>	<code>Bash(...)</code> , <code>Edit(...)</code> , <code>Read(...)</code> , <code>Write(...)</code> , <code>WebFetch(...)</code>
Cursor	<code><project>/cursor/cli.json</code>	<code>Shell(...)</code> , <code>Read(...)</code> , <code>Write(...)</code> , <code>WebFetch(...)</code> , <code>Mcp(...)</code>

Starter templates live in `docs/superpowers/reference/`. Example for Claude Code:

```
{
  "permissions": {
    "allow": ["Read(*)", "Edit(*)", "Bash(pytest:*)"],
    "deny": ["Bash(git push:*)", "Bash(rm -rf:*)", "Bash(sudo:*)",
             "Bash(curl:*)", "Bash(wget:*)"]
  }
}
```

3 Restart and test with something harmless

Re-run `./setup-agent.sh`, open a session in an allow-listed project, and send a read-only prompt. Check the result in `/jobs`.

Why the dialect matters

Cursor has no `Bash(...)` and Claude Code has no `Shell(...)`. Because `cursor-agent --force` allows everything not explicitly denied, a Claude-style profile pasted into `.cursor/cli.json` would deny *nothing* while looking configured. AI Remote therefore rejects wrong-dialect profiles and fails the job instead of running unconstrained. Also: the shipped templates are starting points — replace broad `Read(*)` / `Edit(*)` grants with the smallest set each project needs.

To disable remote execution entirely, leave `AI_REMOTE_ALLOWED_PROJECTS` empty — the app then works as a read-only history viewer.

Environment variables

All settings live in one repo-root `.env` (copy `example.env`). The backend reads it through Docker Compose; the agent via `setup-agent.sh` at install time.

VARIABLE	DEFAULT	PURPOSE
<code>API_KEY</code>	<i>required, ≥32</i>	Shared secret: agent bearer token and login password. <code>openssl rand -hex 32</code> .
<code>SECRET_KEY</code>	<i>required, ≥32</i>	Signs the browser session cookie.
<code>PORT</code>	8000	Local port bound on loopback.
<code>DATABASE_PATH</code>	app.db	SQLite file location.
<code>SESSION_COOKIE_HTTPS_ONLY</code>	true	Marks the cookie <code>Secure</code> . <code>run.sh</code> sets false for local http.
<code>LOCAL_HOME_DIR</code>	/Users/yourname	Your Mac's home directory, for <code>~</code> expansion in the path filter.
<code>CHAT_HISTORY_PAGE_SIZE</code>	10	Messages per "load more" click.
<code>AI_REMOTE_BACKEND_URL</code>	—	Where the agent sends sync and job requests.
<code>AI_REMOTE_INTERVAL_SECONDS</code>	60	Default (idle) poll interval.
<code>AI_REMOTE_ACTIVE_INTERVAL_SECONDS</code>	10	Poll interval during an active window.
<code>ACTIVE_INTERVAL_DURATION_MIN</code>	5	How long an active window lasts after phone activity.
<code>AI_REMOTE_ALLOWED_PROJECTS</code>	empty = off	Comma-separated absolute paths where remote commands may run. Must match on both sides.
<code>TRUSTED_PROXY_HOPS</code> / <code>TRUSTED_PROXIES</code>	0 / empty	Trusted reverse-proxy configuration for login throttling. Set together, or the backend refuses to start.
<code>ENABLE_API_DOCS</code>	false	Serve <code>/docs</code> and <code>/openapi.json</code> .
<code>AI_REMOTE_AGENT_LABEL</code>	com.example.ai-remote-agent	launchd service label (reverse-DNS style).
<code>REGISTRY</code>	—	Container registry for <code>build-and-push.sh</code> .
<code>SSH_HOST</code> / <code>SSH_PATH</code>	— / /opt/ai-remote-backend	Target for the SCP deploy.
<code>PLESK_HOST</code>	—	SSH target for the Plesk deploy path.

Stack, structure, and contributing

Backend

Python 3.12, FastAPI, Uvicorn, Jinja2 templates, SQLite with FTS5, `markdown` + `pygments` + `bleach`, itsdangerous signed cookies. Packaged as a Docker image.

Agent

Plain Python with a single dependency (`httpx`). Reads Claude Code `.jsonl` transcripts and Cursor's local state; runs as a launchd user service.

Repository layout

```
backend/    FastAPI app: auth, chat list/detail/search, job queue, SQLite + FTS5
agent/      Local polling agent: session scanning, command execution, permission checks
docs/       Design specs and implementation plans for every feature
*.sh        run, agent install/update, and deploy scripts
```

Testing

```
./run.sh --test          # both suites from the repo root
cd backend && .venv/bin/pytest -v
cd agent && .venv/bin/pytest -v
```

Where contributions help most

- **More languages** — the UI ships in English and German (`backend/app/i18n.py`); further locales are one dictionary each.
- **Hardening items from `SECURITY.md`** — non-root container, per-session revocation, CSRF tokens, job retention, running the dependency audit in CI.
- **More tools and platforms** — additional agent CLIs, Linux and Windows agent services.
- **Docs and deploy recipes** — reverse-proxy examples, Compose-with-proxy templates.

FAQ

Does my chat history leave my network?

It is uploaded to *your* backend, which you host. There is no third-party service in the loop, but the backend stores chat content in plain SQLite — protect the server and the `data/` volume accordingly.

Can it modify my existing sessions?

Syncing is read-only. Remote prompts are executed by the official `claude` / `cursor-agent` CLIs, which then write to their own session files as usual.

What if my phone is lost?

Rotate `API_KEY` (and `SECRET_KEY`) and redeploy — rotating either invalidates every existing session. Use the kill switch first if you can still reach the UI.

GET STARTED

Try it in two minutes.

Clone, run `./run.sh`, and see your own sessions on your phone.

Repository	<a href="https://github.com/<your-org>/ai-remote-management">https://github.com/<your-org>/ai-remote-management
Documentation	README.md · SECURITY.md · agent/README.md
Issues	<a href="https://github.com/<your-org>/ai-remote-management/issues">https://github.com/<your-org>/ai-remote-management/issues
License	MIT